

Instruction-Level Parallelism

When programmers hear the word *parallelism*, they mostly think about *multi-core parallelism*, the practice of explicitly splitting a computation into semi-independent *threads* that work together to solve a common problem.

This type of parallelism is mainly about reducing *latency* and achieving *scalability*, but not about improving *efficiency*. You can solve a problem ten times as big with a parallel algorithm, but it would take at least ten times as many computational resources. Although parallel hardware is becoming **ever more abundant** and parallel algorithm design is becoming an increasingly important area, for now, we will limit ourselves to considering only a single CPU core.

But there are other types of parallelism, already existing inside a CPU core, that you can use *for free*.

Instruction Pipelining

To execute *any* instruction, processors need to do a lot of preparatory work first, which includes:

- **fetching** a chunk of machine code from memory,
- **decoding** it and splitting into instructions,
- **executing** these instructions, which may involve doing some **memory** operations, and
- **writing** the results back into registers.

This whole sequence of operations is *long*. It takes up to 15-20 CPU cycles even for something simple like `add`-ing two register-stored values together. To hide this latency, modern CPUs use *pipelining*: after an instruction passes through the first stage, they start processing the next one right away, without waiting for the previous one to fully complete.



Pipelining does not reduce *actual* latency but functionally makes it seem like if it was composed of only the

execution and memory stage. You still need to pay these 15-20 cycles, but you only need to do it once after you've found the sequence of instructions you are going to execute.

Having this in mind, hardware manufacturers prefer to use *cycles per instruction* (CPI) instead of something like "average instruction latency" as the main performance indicator for CPU designs. It is a [pretty good metric](#) for algorithm designs too, if we only consider *useful* instructions.

The CPI of a perfectly pipelined processor should tend to one, but it can actually be even lower if we make each stage of the pipeline "wider" by duplicating it, so that more than one instruction can be processed at a time. Because the cache and most of the ALU can be shared, this ends up being cheaper than adding a fully separate core. Such architectures, capable of executing more than one instruction per cycle, are called *superscalar*, and most modern CPUs are.

You can only take advantage of superscalar processing if the stream of instructions contains groups of logically independent operations that can be processed separately. The instructions don't always arrive in the most convenient order, so, when possible, modern CPUs can execute them *out of order* to improve overall utilization and minimize pipeline stalls. How this magic works is a topic for a more advanced discussion, but for now, you can assume that the CPU maintains a buffer of pending instructions up to some distance in the future, and executes them as soon as the values of its operands are computed and there is an execution unit available.

An Education Analogy

Consider how our education system works:

1. Topics are taught to groups of students instead of individuals as broadcasting the same things to everyone at once is more efficient.
2. An intake of students is split into groups led by different teachers; assignments and other course materials are shared between groups.
3. Each year the same course is taught to a new intake so that the teachers are kept busy.

These innovations greatly increase the *throughput* of the whole system, although the *latency* (time to graduation for a particular student) remains unchanged (and maybe increases a little bit because personalized tutoring is more effective).

You can find many analogies with modern CPUs:

1. CPUs use SIMD parallelism to execute the same operation on a block of different data points (comprised of 16, 32, or 64 bytes).
2. There are multiple execution units that can process these instructions simultaneously while sharing other CPU facilities (usually 2-4 execution units).
3. Instructions are processed in pipelined fashion (saving roughly the same number of cycles as the number of years between kindergarten and PhD).

In addition to that, several other aspects also match:

- Execution paths become more divergent with time and need different execution units.
- Some instructions may be stalled for various reasons.
- Some instructions are even speculated (executed ahead of time), but then discarded.
- Some instructions may be split in several distinct micro-operations that can proceed on their own.

Programming pipelined and superscalar processors presents its own challenges, which we are going to address in this chapter.